

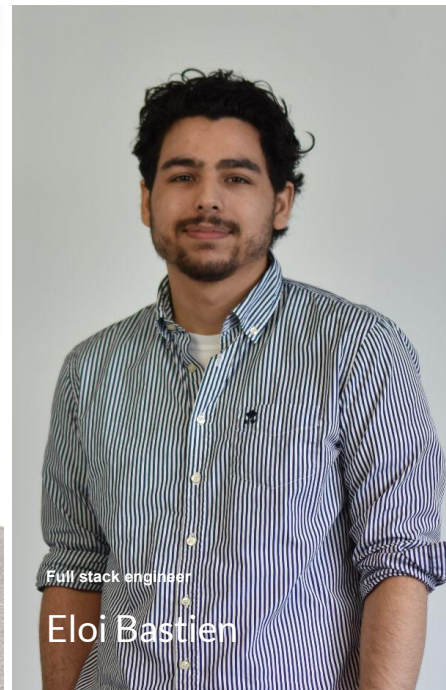


Agent to Agent based troubleshooting

Automate stuff with agents



The team



TOC

Where we started

Deliverables

MCP

Agents and A2A

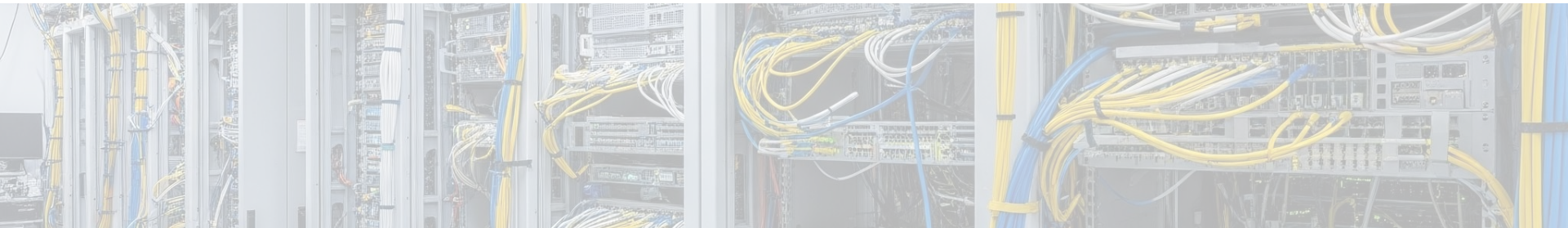
Roadmap

Future



Situation at start of 2026

- Working vector search in development version of WFO
- An experimental agent implementation in orchestrator-core
- An experimental chat implementation in the WFO GUI



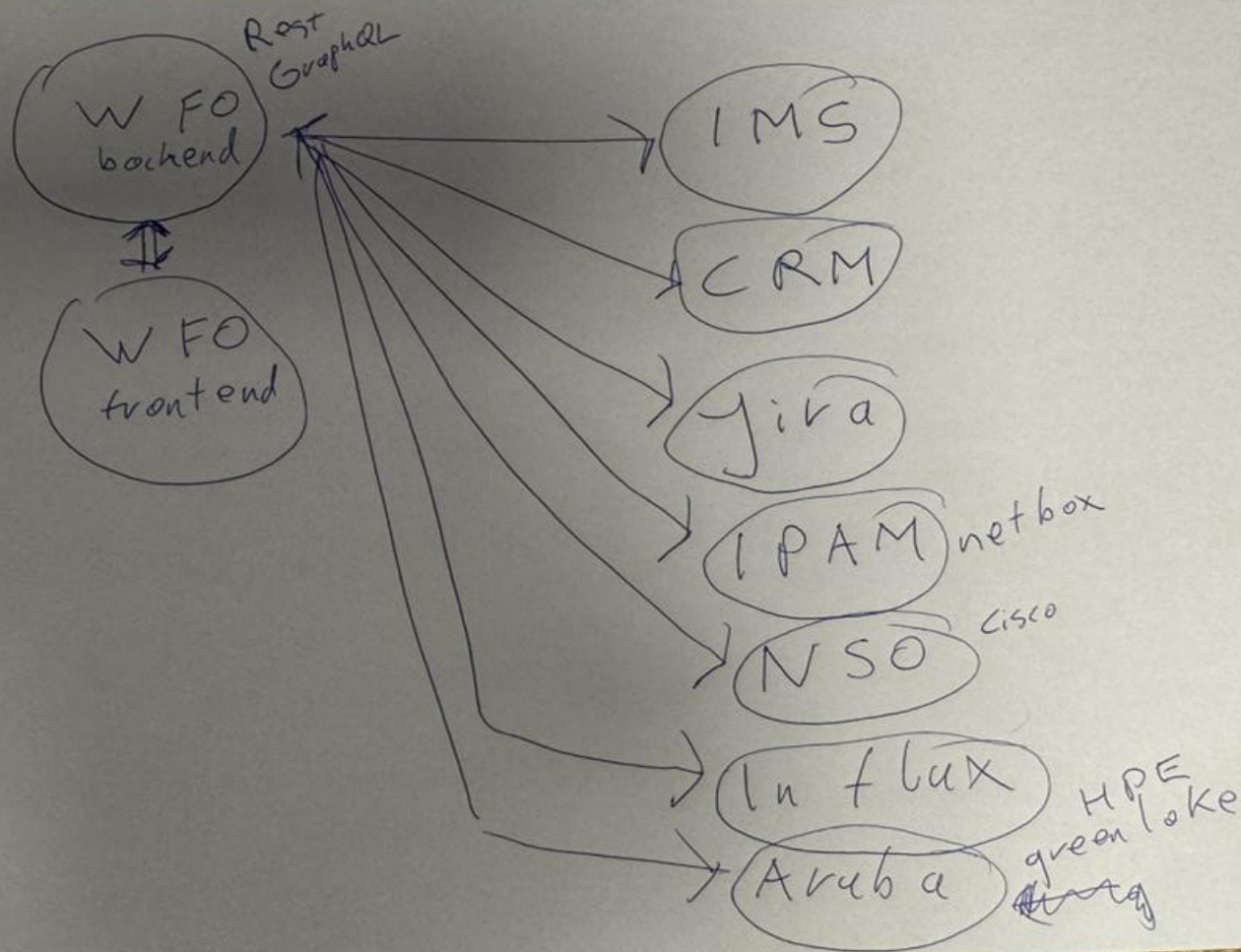
Main goals:

Deliver a *constellation of AI agents* that can troubleshoot incidents and automate a range of use cases.

GOAL: Reduce the workload on first-line helpdesk staff for the fixed network and should also improve the quality and speed of incident response.

Deliverables:

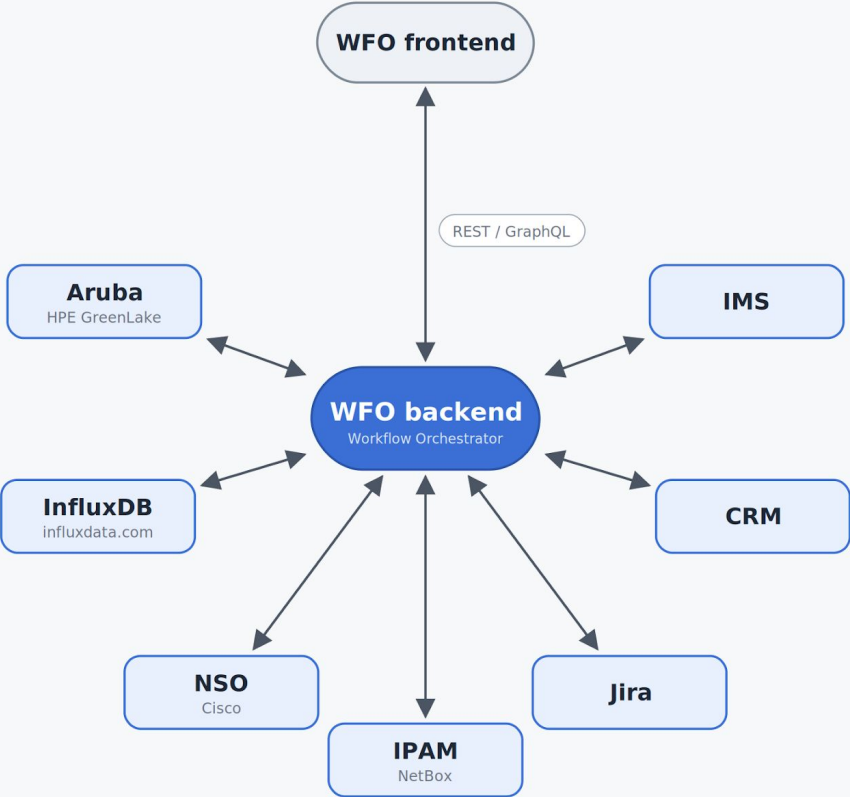
- Build the Agent-to-Agent interface in the Workflow Orchestrator
- Upgrade the current Agent to a Graph Agent
- Add observability: see what agents actually say to each other
- Pick and set up an Agent-to-Agent system that likes Kubernetes
- Build AI Agents for network troubleshooting and resource retrieval
- Build the MCP servers those agents run against
- Document the architecture and code
- Spread the word: talks, workshops, meetings



WFO integration landscape

Current situation

Simplified SURF stack





FIRST: What is an agent?

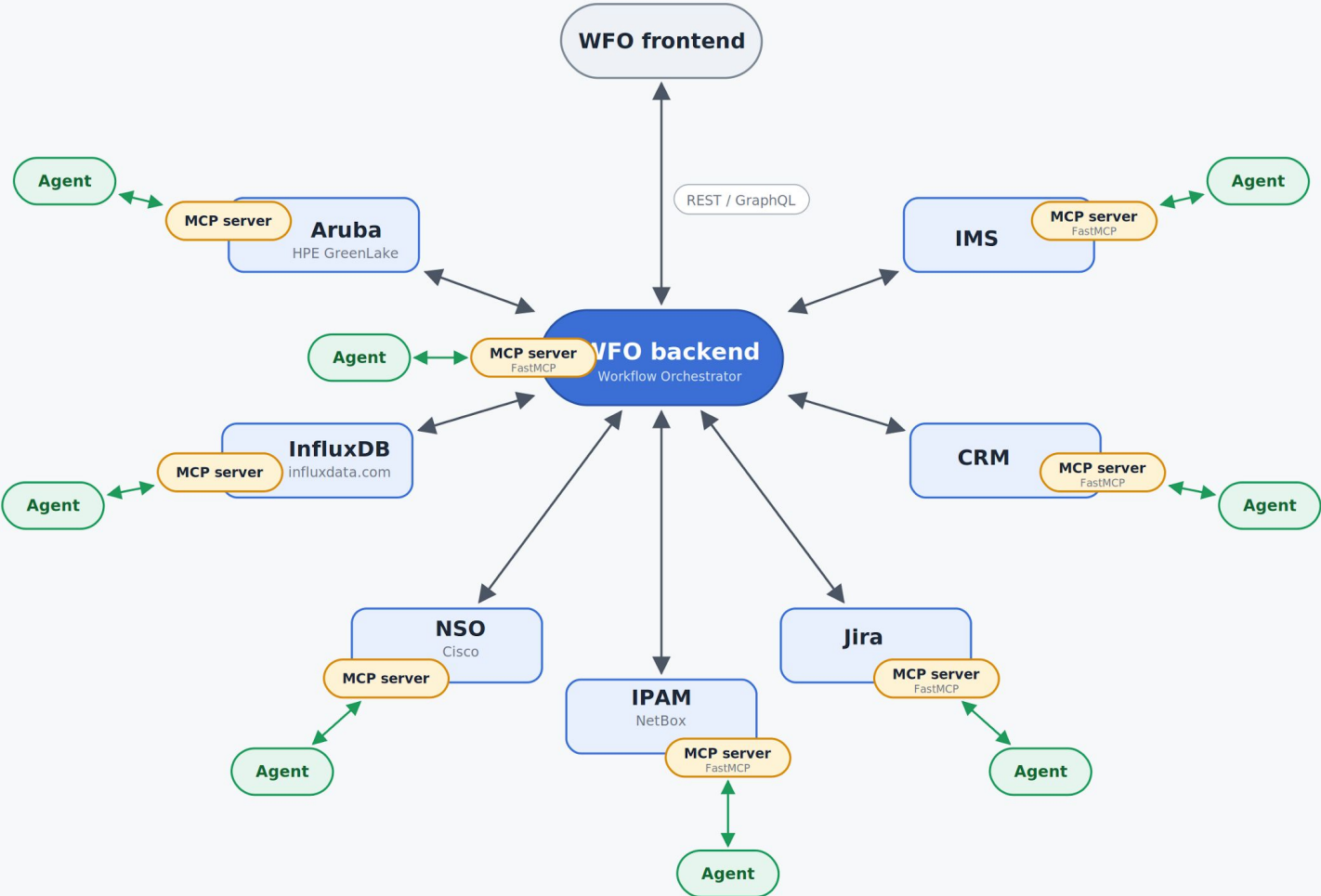
A system prompt, running on a model of choice, that has access to a specified set of MCP tools.

It's mostly stateless and remembers only one session.

WFO integration landscape

Target situation: every system exposes an MCP server, each with its own agent

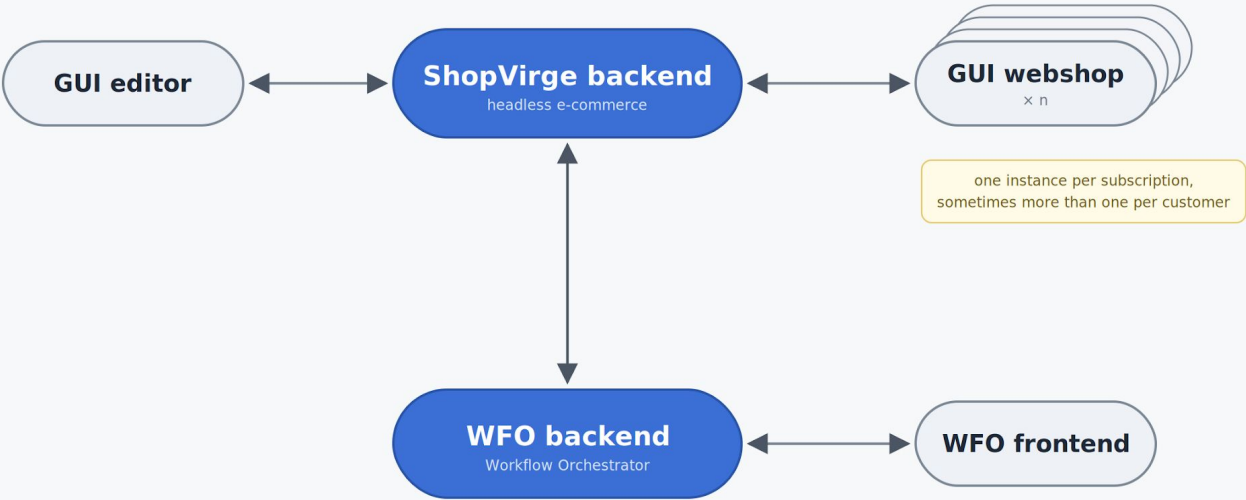
Simplified SURF stack



ShopVirge and WFO landscape

Current situation

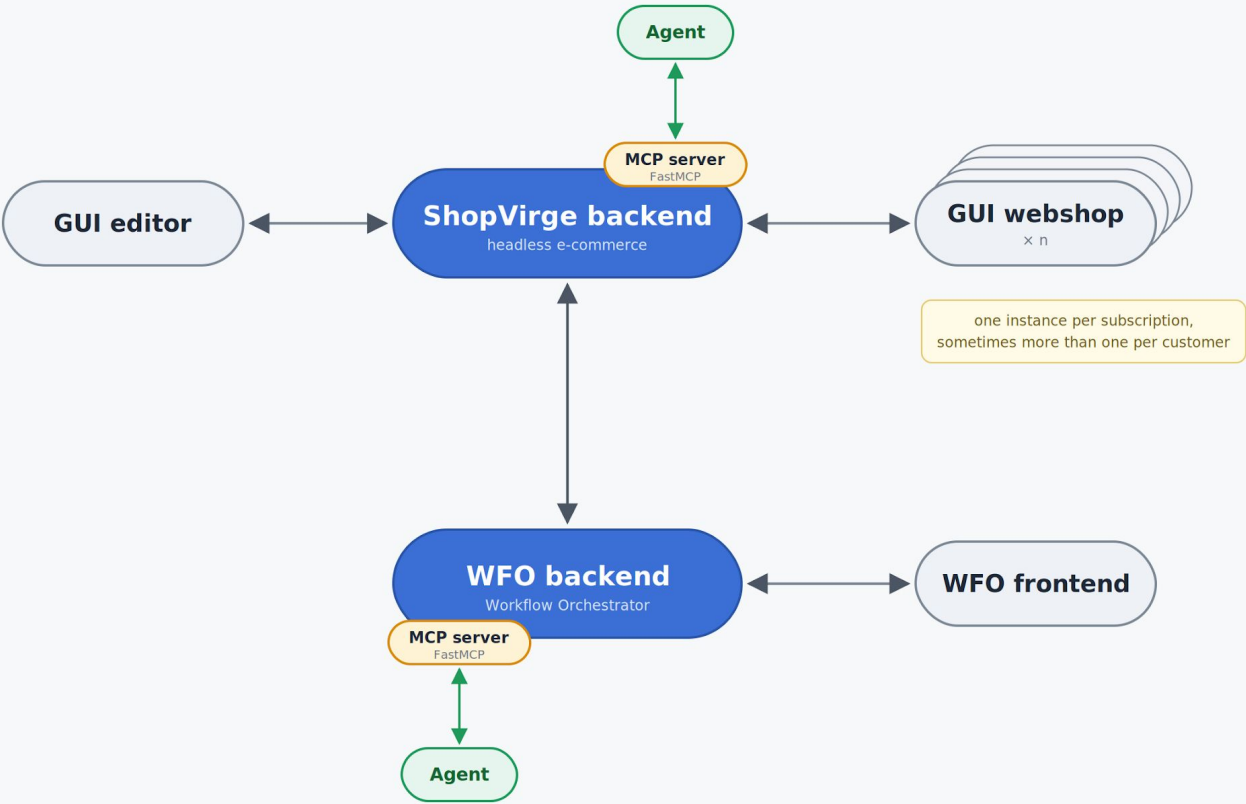
Simplified Virge stack



ShopVirge and WFO landscape

Target situation: both backends expose an MCP server (FastMCP), each with its own agent

Simplified Virge stack



MCP servers

An MCP (Model Context Protocol) server is a small program that exposes tools, data, or prompts to an AI model through a standardized interface, so the model can do things like query a database, read files, or call an API.

Think of it as a universal adapter: any AI client that speaks MCP can plug into any MCP server without custom integration code for each pairing.



MCP Implementation choices / challenges

1

Stay close to the REST /
OpenAPI spec?

3

For FastAPI we recommend
FastMCP

2

Determining the level of
documentation and: share
where possible with
OpenAPI docs

4

Authorisation is shared with
the REST endpoints

23 MCP tools, generated from FastAPI routes

Workflows & processes

list_workflows
get_workflow_by_id
get_workflow_form
create_workflow **write**
resume_workflow_process **write**
abort_workflow_process **destructive**
get_process_status
list_recent_processes
get_process_status_counts

Product catalog

list_products
get_product
get_product_block
get_resource_type

Subscriptions

list_subscriptions
get_subscription_details
get_subscription_domain_model
get_subscription_available_workflows

AI / hybrid search

search
aggregate
discover_filter_paths
get_valid_operators
resolve_entity
export_query

20 / 23

tools are read-only; 2 write, 1
destructive

Tag a route, get a tool

Routes tagged **agent-exposed** become tools.
The operation_id is the name, the docstring
the description, the Pydantic model the
schema.

Same auth as REST

Calls run in-process through FastAPI with the
bearer token forwarded, so
Depends(authorize) applies.

Opt-in

Install the [mcp] extra and set
MCP_ENABLED=True. Served over streamable
HTTP.



Kagent

Lets you define agents as Kubernetes custom resources, and a controller then spins up whatever is needed to run them inside your cluster

Libre chat

A self-hosted chat GUI whose built-in agent builder lets you combine any LLM model with MCP tools, file search and code execution into reusable agents without writing code.

Slack

It acts as the front end and event source, so agents built as Slack apps can be triggered by messages and mentions, read channel context, and reply or take actions right where the team already works.

Docker agent

Formerly cagent, it runs single or multi-agent teams from a declarative YAML file. lets you push and pull agents through OCI registries like container images.

Hermes

Self-hosted agent that runs as a long-lived process with persistent memory and self-created skills, reachable from the terminal or through a messaging gateway such as Telegram, Discord or Slack.



Kagent vs Docker Agent

Kagent

- Kubernetes-native framework: agents, model configs and tool servers are custom resources managed with kubectl
- A controller watches those resources and spins up the pods that run each agent inside the cluster
- Ships MCP tools for Kubernetes, Helm, Istio, Argo, Prometheus and Grafana, so agents can operate infrastructure
- Works with OpenAI, Anthropic, Azure, Vertex AI, Ollama or any model behind an AI gateway
- Built for DevOps and platform teams: observability via OpenTelemetry, a web UI and a CLI

Docker Agent

- Formerly cagent; a Docker CLI plugin bundled with Docker Desktop, started with docker agent run
- Agents are defined declaratively in a YAML file: model, instructions and toolsets, no code needed
- Multi-agent teams where a root agent delegates tasks to specialist sub-agents
- Tools via MCP, including containerized servers from the Docker MCP Gateway and catalog
- Agents are packaged as OCI images, so you push and pull them from Docker Hub like any container
- As a server, you can expose any agent defined in your YAML over A2A. **experimental**

LibreChat

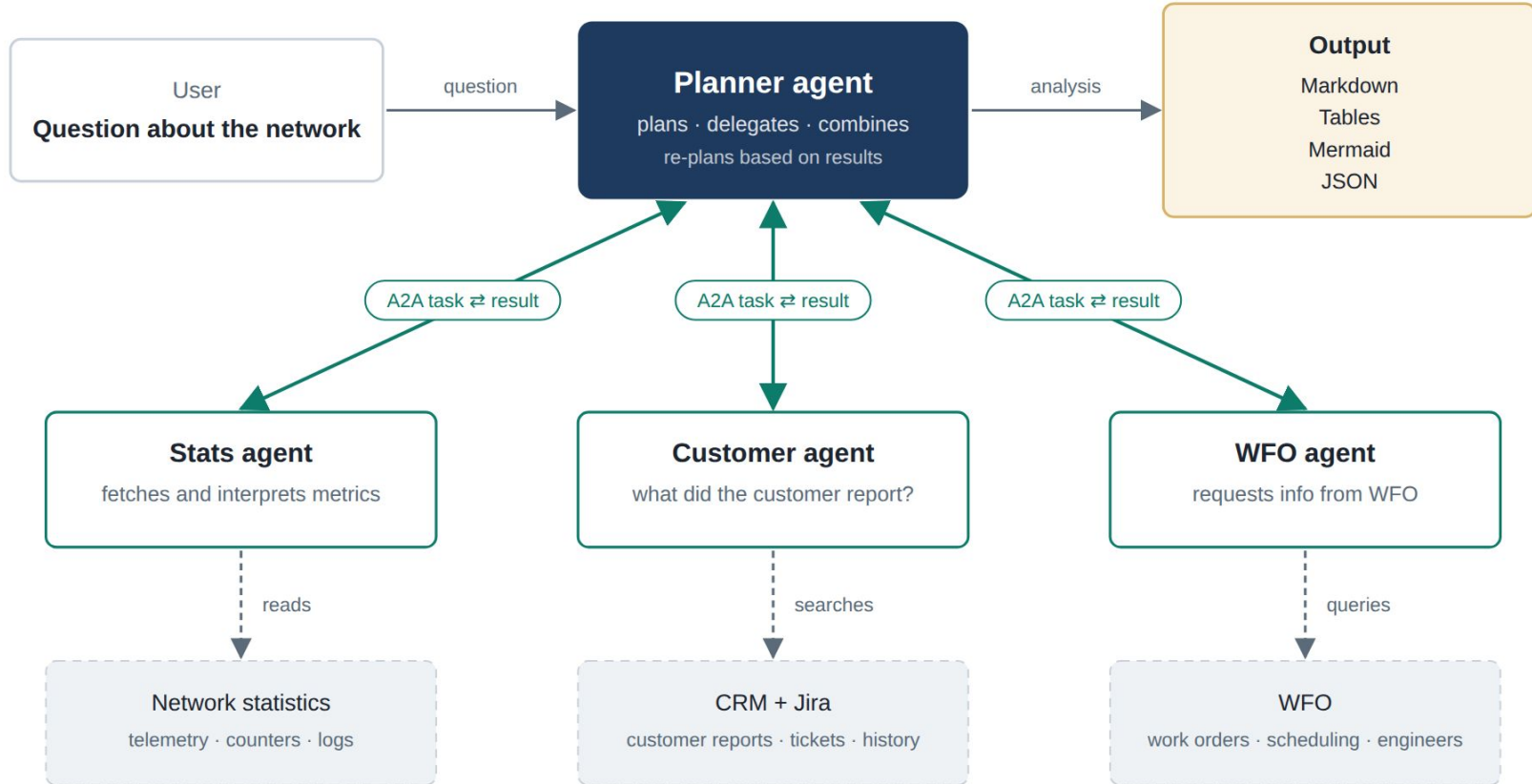
Agents and tools

- No-code agent builder: combine a model, instructions and tools into a reusable assistant
- Connects to any MCP server, so agents can use external tools and data
- Skills: reusable instruction bundles that agents can pick up
- Sandboxed code interpreter for Python, Node.js, Go, Java, C/C++ and more
- A2A (Agent-to-Agent protocol): not native yet, a client PR was proposed; today via an A2A-to-MCP bridge such as agentgateway

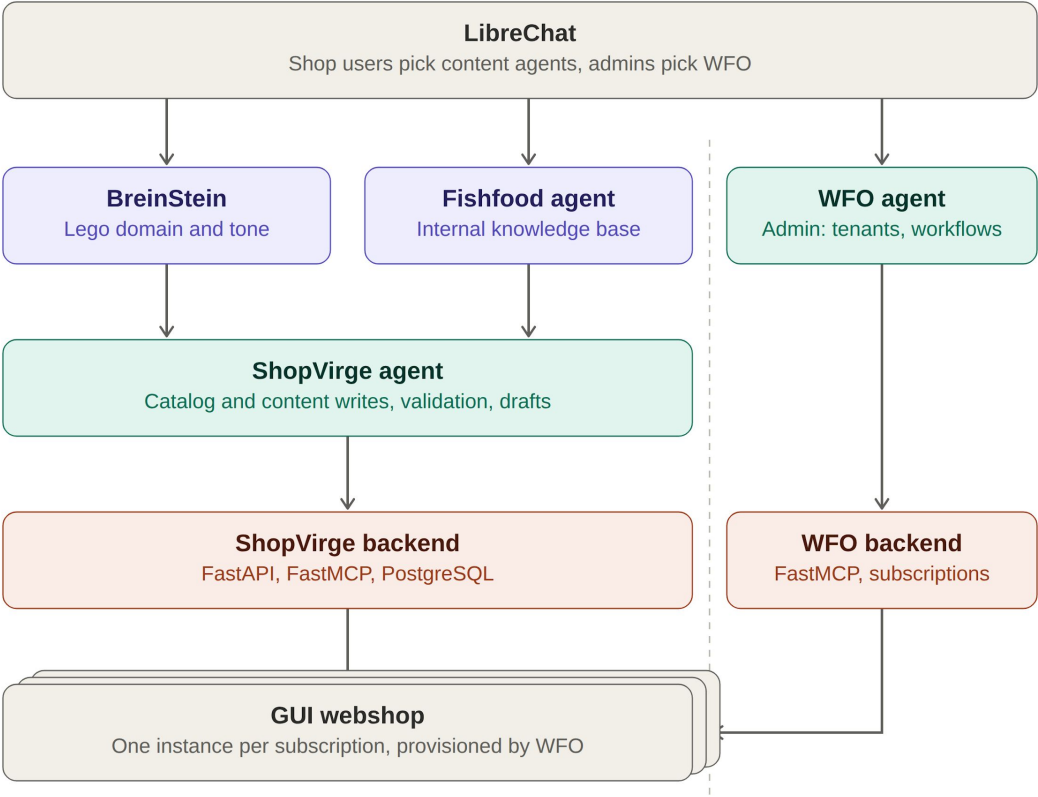
Platform

- Multi-provider: OpenAI, Anthropic, Google, Azure, AWS Bedrock, Mistral, Ollama and OpenAI-compatible endpoints
- Web search and chat with uploaded files across providers
- Multi-user with OAuth2, LDAP and email login, moderation and token limits
- Artifacts: renders React, HTML and Mermaid output live in the chat
- Self-hosted via Docker with an admin panel for users and configuration

End result agent network troubleshooter



End result for the ShopVirge content agents



Left: content lane for shop users. Right: admin lane. Coral: backends with MCP servers.



Demo's



Evals



When one of the agents is changed this could have a negative effect on the whole constellation.

Evals make agent quality measurable

01

Build a task suite

Real tasks from your domain, each with a known good outcome to check against.

02

Run every task often

Agents are nondeterministic, so run several trials and report pass rates, not single wins.

03

Grade the outcome

Code checks where you can, model graders with a rubric where you can't, humans to spot check.

04

Compare and gate

Score every prompt, model or tool change against a baseline and block regressions.

Track per run

Task success rate

Cost per task

Latency

Steps and tool calls



Roadmap

- Improve agent performance with more evals
- More docs
- Autorisation
- Fill out forms automatically
- Autodiscovery



Thank you.





How Virge uses WFO

Using workflows to facilitate eCommerce and Content generation and provide consultants to help people integrate WFO





Nationaal Cyber Security Centrum
Ministerie van Veiligheid en Justitie





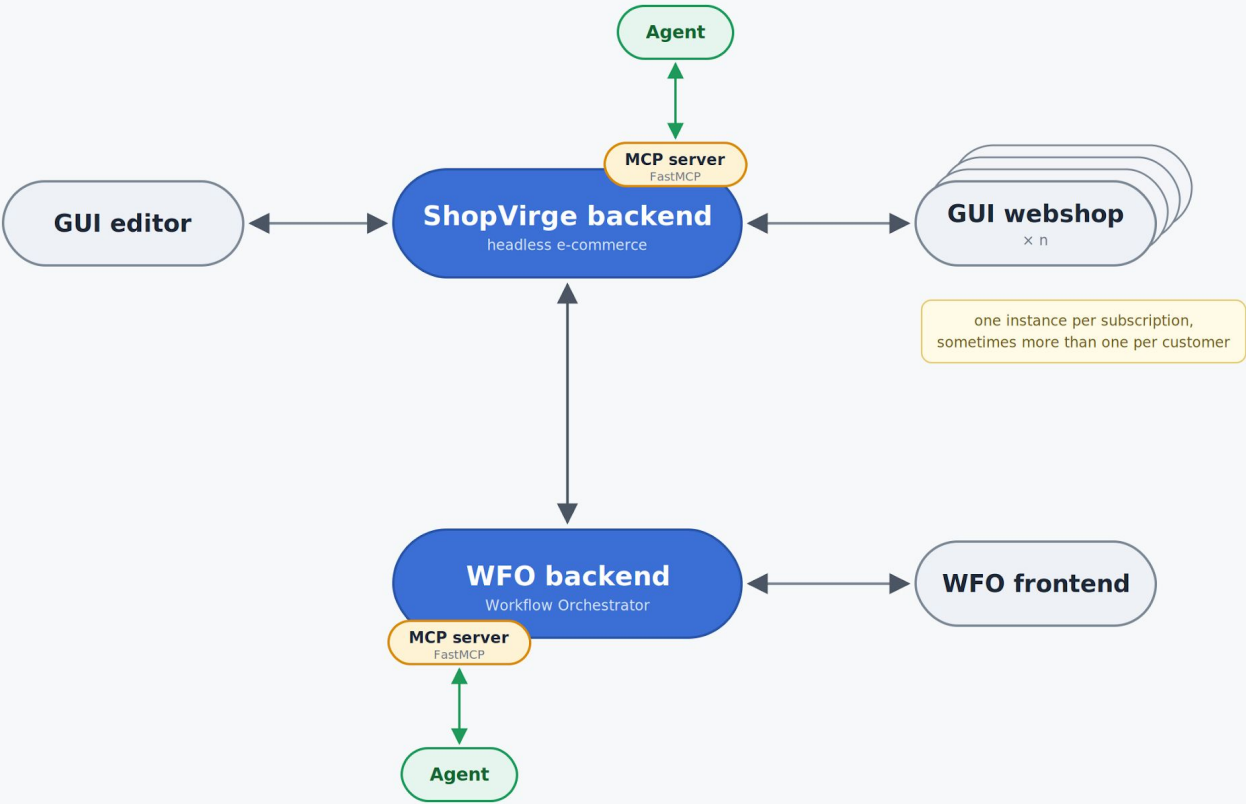
ShopVirge use cases

- Manage lifecycle of Shops
- B2C via an external form
- Generate content (legacy)
- Validate against drift

ShopVirge and WFO landscape

Target situation: both backends expose an MCP server (FastMCP), each with its own agent

Simplified Virge stack





Hosting the WFO on AWS without k8s

OLD

- Backend on AWS Apprunner
- Celery worker via Docker on TransIP VM
- Frontend on vercel (hard fork)
- Redis at Upstash
- No scheduler

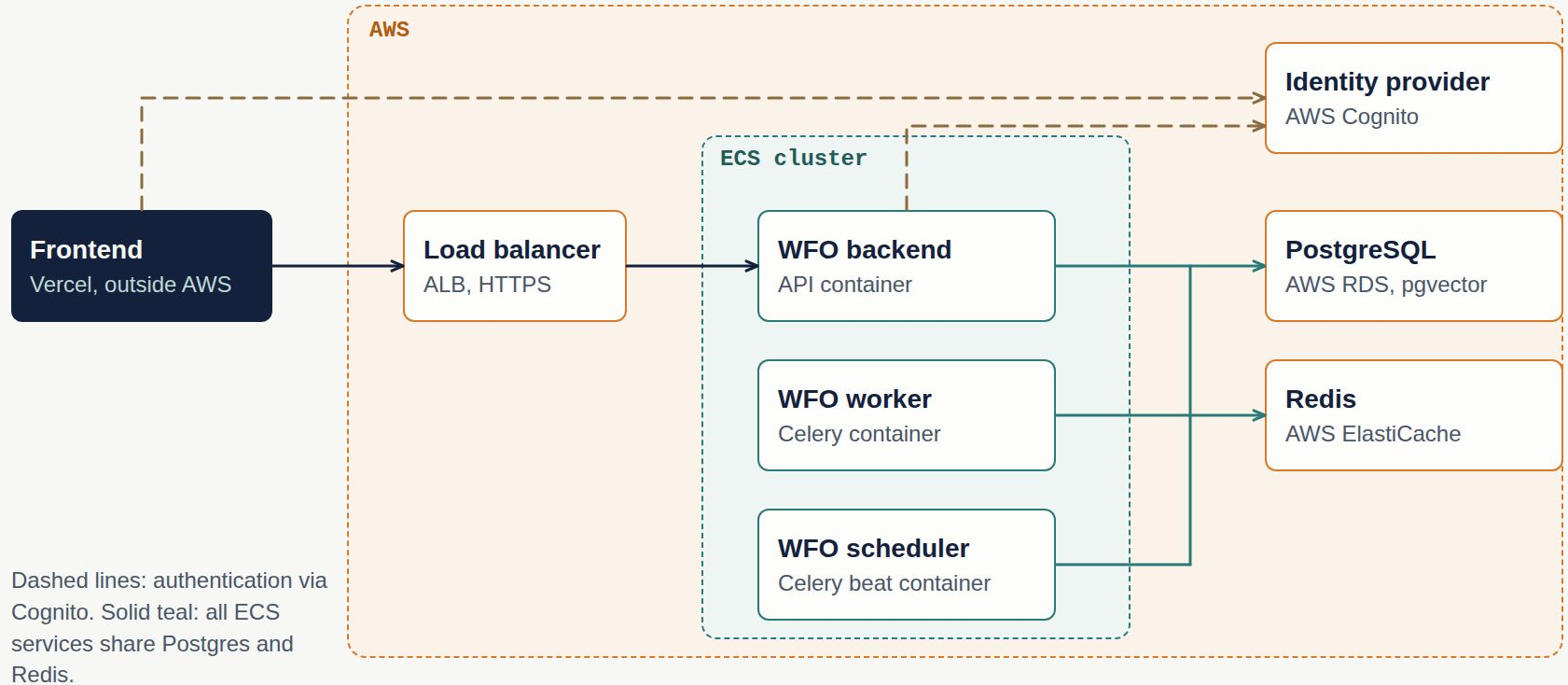
[Link](#)

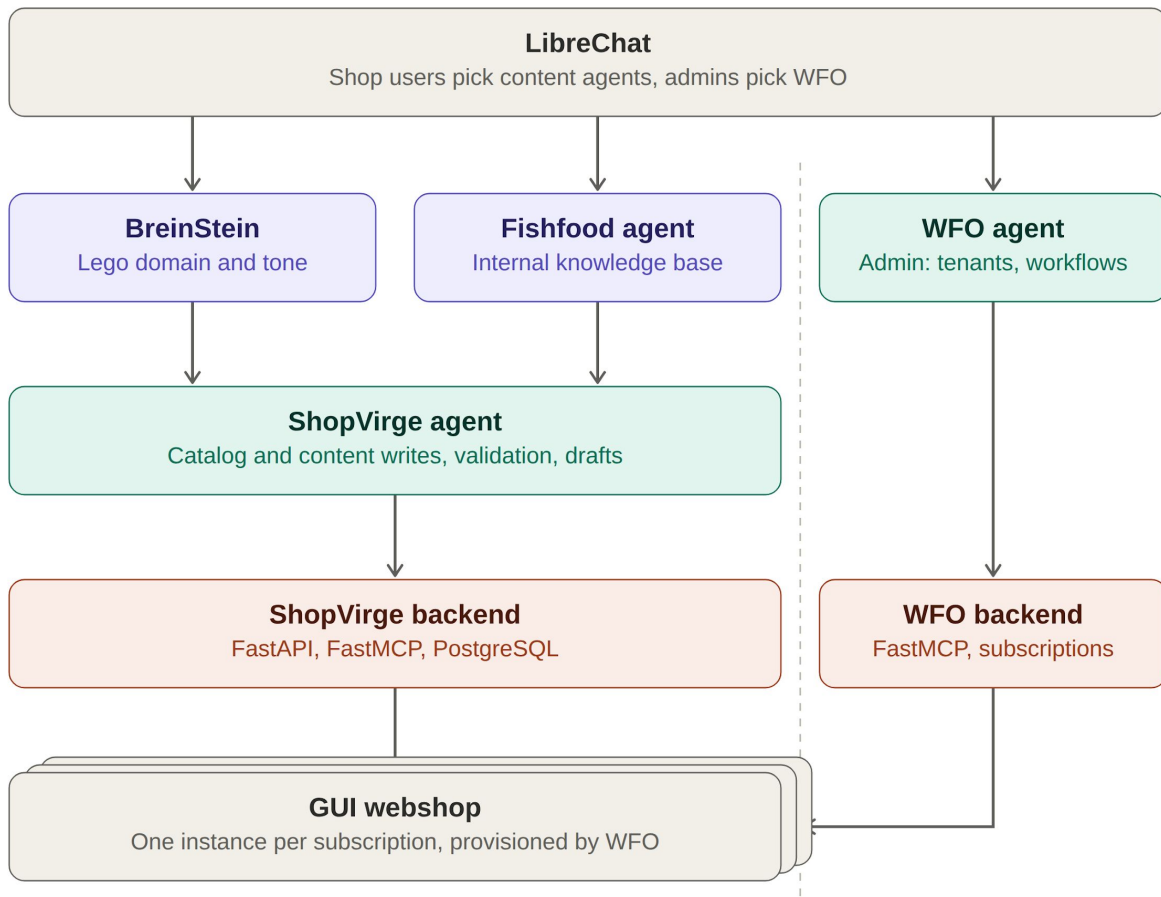
NEW

- Backend on AWS ECS
- All AWS goodies are provisioned via Tofu
- Celery is now also a container on AWS
- Scheduler is now also a container on AWS
- Frontend on Vercel (with a claude skill to sync)

[Link](#)

WFO on AWS ECS: architecture overview





Left: content lane for shop users. Right: admin lane. Coral: backends with MCP servers.



Thank you.

